

Project Report: Hospital Management System

Author

- **Name:** Soumalya Ghosh
- **Roll Number:** 24f2006296
- **Email:** 24f2006296@ds.study.iitm.ac.in
- **About Me:** I am a second-year BS Data Science student at IIT Madras , interested in building well-structured applications that deliver practical solutions.

Description

This project is a web-based **Hospital Management System** designed to simplify the interaction between patients, doctors, and hospital administrators. It allows patients to book appointments online, doctors to manage their schedules and treatments, and administrators to oversee the entire hospital's operations efficiently.

AI/LLM Declaration

I have used Artificial Intelligence (Gemini 3) tools for conceptual understanding & fixing errors. The breakdown of usage is as follows:

1. **Concept Clarification:** I used the AI to learn & understand specific concepts like Routing Logic and Database Design. However, the final code structure, routing logic implementation were executed by me.
2. **Debugging:** AI was used to debug syntax errors and fix database session issues during development.

Technologies Used

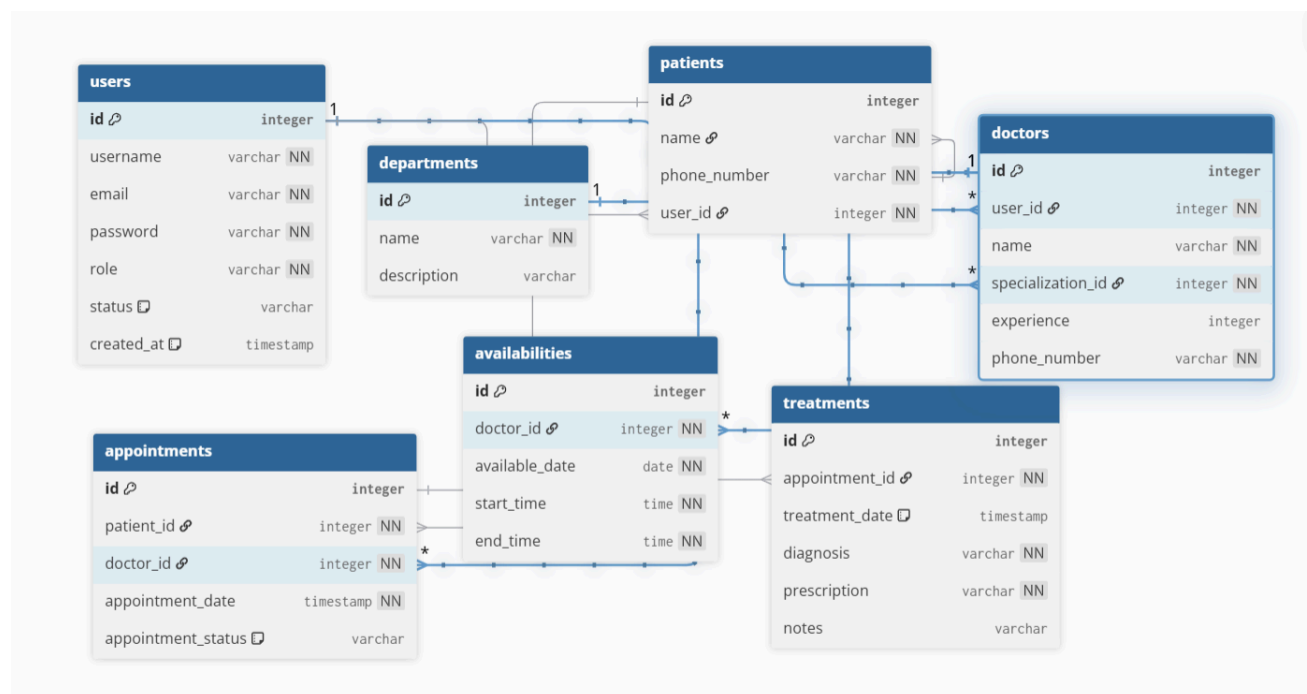
1. **Python (Flask):** Used as the backend framework to handle server logic, routing, and database interactions. It was chosen for its simplicity and flexibility.
2. **Flask-SQLAlchemy:** Used as an Object Relational Mapper (ORM) to interact with the database using Python classes instead of writing raw SQL queries.
3. **SQLite:** Used as the database engine to store user data, appointments, and medical records. It is lightweight and requires no separate server configuration.
4. **HTML & CSS:** Used for the frontend to create a user-friendly and responsive interface with a professional "Medical Green & White" theme.
5. **Jinja2 Templating:** Used to render dynamic data (like patient names and appointment lists) directly into the HTML pages.

DB Schema Design / ER Diagram

The database is designed using a relational model to ensure data integrity and reduce redundancy.

- **User:** The central table for authentication. Stores username, email, password, role (Admin/Doctor/Patient), and status.
- **Patient:** Linked to the User table. Stores patient-specific details like name, phone_number.
- **Department:** Stores medical specializations (e.g., Cardiology) to categorize doctors.
- **Doctor:** Linked to User and Department. Stores name, experience, phone, and availability details.
- **Appointment:** The core transaction table. Links a Patient and a Doctor for a specific date and time. It tracks the status (Booked/Completed/Cancelled).
- **Treatment:** Linked to Appointment. Stores the medical outcome of a visit, including diagnosis, prescription, and notes.
- **Availability:** Stores the dates and times when a doctor is available to see patients.

Reasoning: This normalized design separates login credentials (User) from profile details (Doctor/Patient), enhancing security and organization.



API Design (Internal Routes)

Since this is a server-side rendered application, internal Flask routes function as the API endpoints:

- **Auth Routes:** /login, /register, /logout handle user authentication and session management.
- **Patient Routes:**
 - /patient/book_appointment (POST): Creates a new appointment record.
 - /patient/dashboard (GET): Fetches doctors and upcoming appointments.
- **Doctor Routes:**
 - /doctor/treatment (POST): Saves diagnosis and prescription data.
 - /doctor/update_status (GET): Updates appointment status (Completed/Rejected).
- **Admin Routes:**
 - /admin/add_doctor (POST): Creates new doctor profiles and user accounts programmatically.

Architecture and Features

Architecture:

The project follows the Model-View-Template (MVT) architecture provided by Flask.

- **Model:** Defined in app.py using SQLAlchemy classes (User, Doctor, etc.) to manage data.
- **View (Controller):** The routes in app.py handle the logic, process user input, and query the database.
- **Template:** The templates/ folder contains HTML files that display data to the user. The static/ folder holds CSS and images.

Features Implemented:

1. **Role-Based Access Control:** Secure login for Admins, Doctors, and Patients with specific dashboards for each.
2. **Appointment Management:** Patients can search for doctors and book slots. Doctors can accept, reject, or complete these appointments.
3. **Medical History:** Doctors can issue prescriptions, and patients can view their past treatment history details.
4. **Admin Administration:** Full control to add doctors, manage patients, and view hospital statistics.

Video

[https://drive.google.com/file/d/1KdqkIB_byUpweFZwM-At7yyjy2exfLgr/view?usp=sharing]